

```

[1]: # =====
# Independent SageMath / MPFI validation
# =====
#
# We certify:
#
# 1.  $K(c, X)$  is strictly contained in  $X$ .
# 2.  $\|I - C[DG](X)\|_{\infty} < 1$ .
# 3. Period 2 is impossible.
# 4.  $-2 < \text{trace}(DF^6(p)) < -1$ .
#
# All input decimals are first interpreted as exact rationals.
# Only then are they converted to intervals.
# =====

# -----
# Precision
# -----

# 256 binary bits is about 77 decimal digits.
# This is already more than enough here.
R = RealIntervalField(256)

# Use exact rational decimals for ALL constants supplied by hand.
from fractions import Fraction
from sage.all import QQ

def exact_decimal(s):
    """Convert a decimal string to an exact Sage rational."""
    if not isinstance(s, str):
        raise TypeError("Pass a string, not a floating-point number.")

    q = Fraction(s)
    return QQ(q.numerator) / QQ(q.denominator)

# -----
# Certified center and box
# -----

cx_Q = exact_decimal(
    "0.
    ↪27474448332000881878050615161797640957394247787842890490043558664838204815059897"
)

cy_Q = exact_decimal(

```

```

"0.
↪092588780576427956472961292120881745888037901834466710224563523023949661988119064"
)

r_Q = exact_decimal("1e-30")

# Point intervals enclosing the exact decimal rationals.
c = vector(R, [
    R(cx_Q),
    R(cy_Q),
])

# [-r, r]^2
D = vector(R, [
    R(-r_Q, r_Q),
    R(-r_Q, r_Q),
])

#  $X = c + [-r, r]^2$ 
X = c + D

# -----
# Fixed preconditioner C
# -----
#
# Important:
# We do NOT compute C by interval inversion here.
#
# These are fixed exact decimal rationals, chosen previously
# as an approximation to  $DG(c)^{-1}$ . Krawczyk does not require
# C to be the exact inverse.
# -----

C_Q = [
    [
        exact_decimal(
            "0.413502121757549070333801035742412075797"
        ),
        exact_decimal(
            "-2.399328010952348941913855704225548696173"
        ),
    ],
    [
        exact_decimal(
            "0.3480268316536689025396153232872661506545"
        ),
    ],
]

```

```

        exact_decimal(
            "-1.413502121757549070333380103574241207580"
        ),
    ],
]

C = matrix(R, [
    [R(C_Q[0][0]), R(C_Q[0][1])],
    [R(C_Q[1][0]), R(C_Q[1][1])],
])

I = identity_matrix(R, 2)

# -----
# One kicked-Harper iterate
# -----

def F_step(z):
    """
    Interval extension of

         $y_1 = y + \sin(2\pi x)$ 
         $x_1 = x + \sin(2\pi y_1)$ 

    Returns an interval vector enclosing  $F(z)$ .
    """

    x = z[0]
    y = z[1]

    two_pi = 2 * R.pi()

    y1 = y + (two_pi * x).sin()
    x1 = x + (two_pi * y1).sin()

    return vector(R, [x1, y1])

# -----
# Jacobian of one iterate
# -----

def DF_step(z):
    """
    Interval enclosure of  $DF(z)$ .

```

```

If

     $b = 2\pi \cos(2\pi x)$ 
     $a = 2\pi \cos(2\pi y_1),$ 

then

     $DF = \begin{bmatrix} 1+a*b & a \\ b & 1 \end{bmatrix}.$ 
    """

x = z[0]
y = z[1]

two_pi = 2 * R.pi()

y1 = y + (two_pi * x).sin()

b = two_pi * (two_pi * x).cos()
a = two_pi * (two_pi * y1).cos()

return matrix(R, [
    [1 + a*b, a],
    [b, 1],
])

# -----
# Simultaneous propagation of  $F^n$  and  $DF^n$ 
# -----

def iterate_with_derivative(z0, n):
    """
    Returns

         $z_n$  containing  $F^n(z_0)$ 
         $M_n$  containing  $DF^n(z_0)$ 

    for an interval starting box  $z_0$ .
    """

    z = z0
    M = identity_matrix(R, 2)

    for k in range(n):

        # DF evaluated at the current interval enclosure.

```

```

    A = DF_step(z)

    # Chain rule:
    #
    #  $D(F \circ \text{current}) = DF(\text{current}) * D(\text{current})$ 
    #
    M = A * M

    # Advance state.
    z = F_step(z)

    return z, M

# =====
# PART 1: residual  $G(c)$ 
# =====

Fc6, DFc6 = iterate_with_derivative(c, 6)

G_c = vector(R, [
    Fc6[0] - c[0] - 3,
    Fc6[1] - c[1],
])

print()
print("=====")
print("1. Interval residual  $G(c)$ ")
print("=====")

print("G1(c) =", G_c[0].str(style='brackets'))
print("G2(c) =", G_c[1].str(style='brackets'))

# =====
# PART 2:  $[DG](X)$ 
# =====

FX6, DFX6 = iterate_with_derivative(X, 6)

#  $G(z) = F^6(z) - z - (3, 0)$ 
#
# Therefore
#
#  $DG(z) = DF^6(z) - I.$ 
#
DG_X = DFX6 - I

```

```

# =====
# PART 3: Krawczyk operator
# =====
#
#  $K(c, X)$ 
#  $= c - C G(c)$ 
#  $+ (I - C [DG](X))(X-c)$ 
#
# Since  $D = X-c$ , it is convenient to compute  $K-c$ :
#
#  $K-c$ 
#  $= -C G(c)$ 
#  $+ (I - C [DG](X)) D.$ 
# =====

E = I - C * DG_X

K_minus_c = -C * G_c + E * D

print()
print("=====")
print("2. Krawczyk image relative to center")
print("=====")

print("(K-c)_1 =", K_minus_c[0].str(style='brackets'))
print("(K-c)_2 =", K_minus_c[1].str(style='brackets'))

print()
print("radius r =", R(r_Q).str(style='brackets'))

# -----
# Rigorous strict-inclusion test
# -----
#
# magnitude() is an UPWARD-ROUNDED upper bound for
#
#  $\max\{|x| : x \text{ in interval}\}.$ 
#
# Sage documents this behavior explicitly.
# -----

k1_mag = K_minus_c[0].magnitude()
k2_mag = K_minus_c[1].magnitude()

```

```

# lower endpoint of R(r) is <= exact r.
# Thus proving mag < lower(R(r)) is certainly sufficient.
r_lower = R(r_Q).lower()

assert k1_mag < r_lower
assert k2_mag < r_lower

print()
print("PASS: K(c,X) is strictly contained in X.")

# =====
# PART 4: infinity norm of I-C[DG](X)
# =====

def interval_matrix_inf_norm_upper(A):
    """
    Rigorous upper bound for the matrix infinity norm:

     $\max_i \sum_j |A_{ij}|$ .

    Each interval absolute value is enclosed rigorously,
    and magnitude() rounds upward.
    """

    row_bounds = []

    for i in range(A.nrows()):

        row_sum = R(0)

        for j in range(A.ncols()):
            row_sum += abs(A[i,j])

        row_bounds.append(row_sum.magnitude())

    return max(row_bounds)

norm_E = interval_matrix_inf_norm_upper(E)

print()
print("=====")
print("3. Contraction / inverse-quality bound")
print("=====")

print("||I - C[DG](X)||_inf <=", norm_E)

```

```

assert norm_E < 1

print("PASS: ||I - C[DG](X)||_inf < 1.")

# =====
# PART 5: exclude torus period two
# =====
#
# If the least period were 2, the lifting argument forces
#
#  $F^2(p) = p + (1, 0)$ .
#
# In particular the second component of  $F^2(p) - p$  would be 0.
#
# We evaluate this difference on the entire certified root box.
# =====

FX2, dummy = iterate_with_derivative(X, 2)

period2_vertical = FX2[1] - X[1]

print()
print("=====")
print("4. Period-two exclusion")
print("=====")

print(
    "(F^2(X)-X)_2 =",
    period2_vertical.str(style='brackets')
)

assert not period2_vertical.contains_zero()

print("PASS: vertical displacement excludes period 2.")

# =====
# PART 6: trace enclosure
# =====

trace6 = DFX6[0,0] + DFX6[1,1]

print()
print("=====")
print("5. Trace enclosure")

```

```

print("=====")

print(
    "trace(DF^6(X)) =",
    trace6.str(style='brackets')
)

# For a rigorous comparison, compare interval endpoints.

assert trace6.lower() > -2
assert trace6.upper() < -1

print("PASS: -2 < trace(DF^6(p)) < -1.")

# =====
# Summary
# =====

print()
print("=====")
print("ALL BASIC INTERVAL CHECKS PASSED")
print("=====")
print()
print("Certified conclusions:")
print(" * unique zero of G in X")
print(" * F^6(p) = p + (3,0)")
print(" * least torus period = 6")
print(" * elliptic trace lies in (-2,-1)")
print()

```

```

=====
1. Interval residual G(c)
=====

```

```

G1(c) = [-
1.061557838792209133352490370975425115281578764796507707847113630553123304444726
e-73 .. 1.0549943506903373554371967435596968116075306045195361339944956159157951
74674323e-73]
G2(c) = [-
1.727665519446643647308539690153219934201755872905478091073991879207226790222644
e-74 .. 1.7113647512989028830781229510121841800112744222176046099597596389072835
73194175e-74]

```

```

=====
2. Krawczyk image relative to center
=====

```

```

(K-c)_1 = [-

```


